

三期多模态机器人动力学建模计算与代码详解

统一的动力学建模部分(六自由度无载具)

☐ 首先要设定参数

```
clear;
clc;
close all;
syms t real
syms m real
syms ix iy iz ixy ixz iyz real
syms xG yG zG real %重心位置
syms xB yB zB real %浮心位置
syms W B real %重力和浮力
syms q1(t) q2(t) q3(t) q4(t) q5(t) q6(t)
syms u(t) v(t) w(t) p(t) q(t) r(t)
syms u_d(t) v_d(t) w_d(t) p_d(t) q_d(t) r_d(t)
```

☐ 其次定义旋转矩阵(绕三轴旋转的旋转矩阵 x\y\z)

```
Rx = [1, 0, 0;
      0, cos(q4(t)), -sin(q4(t));
      0, sin(q4(t)), cos(q4(t))];
Ry = [cos(q5(t)), 0, sin(q5(t));
      0, 1, 0;
      -sin(q5(t)), 0, cos(q5(t))];
Rz = [cos(q6(t)), -sin(q6(t)), 0;
      sin(q6(t)), cos(q6(t)), 0;
      0, 0, 1];
```

☐ 计算组合旋转矩阵并列速度旋转雅可比矩阵 >>机器人运动学可以找到相应速度旋转矩阵由来

Important

理解雅可比矩阵是理解运动学坐标变换的一个关键点。许多教材的旋转矩阵表示可能有所不同，有的同学可能一开始看了机器人学的书籍，再看Fossen的建模书籍，容易把一些概念混淆。其实他们都是一类东西，但是对于AUV,ROV,无人机等涉及NED导航的系，可以直接关注书籍中的设定。

首先在这里详细解释欧拉角。我们在使用欧拉角时，必须严格遵守定义的顺序Z-Y-X。那么雅可比矩阵 J_1 便很容易得知为：

如何理解这个公式？为什么书上既说是从Z-Y-X变换，有时候又说左乘才是相应的变换？

Tip

首先，先搞清楚左乘和右乘的概念。

左乘：说明运动是相对固定坐标系而言的，所谓的左乘>>从“右”往“左”变换意味着对某个向量，某个对象执行从“右”往“左”的变换。比如有一个向量 p ， p 原先的表达是在物体坐标系下的而非固定坐标系， p 左乘变换矩阵 J_1 就是执行从“右”往“左”的变换，则从物体坐标系转换到了固定坐标系，得到 p' 。

右乘：说明运动是相对物体坐标系而言的，它表明运动是相对于“当前物体坐标系的”，我们看 $R_z R_y R_x$ 本身，注意这时候不要想象加上任何物体和向量，就看这个转换本身，其就相当于单位向量 I 乘以这个转换矩阵 $I R_z R_y R_x$ ，也就是单位矩阵 I 经历了右乘的变换。将单位矩阵 I 看作是固定坐标系(惯性坐标系)，那么右乘就是将惯性坐标系变换到了物体坐标系。所以说这是右乘表示的是旋转是相对于当前局部轴进行。如果你非要想象一个向量右乘旋转矩阵，那么它得到的是这个向量不断进行局部旋转后的最终姿态。

$$J_1 = R_z R_y R_x$$

那么接下来我们看Fossen书中对水下机器人坐标系的描述定义。

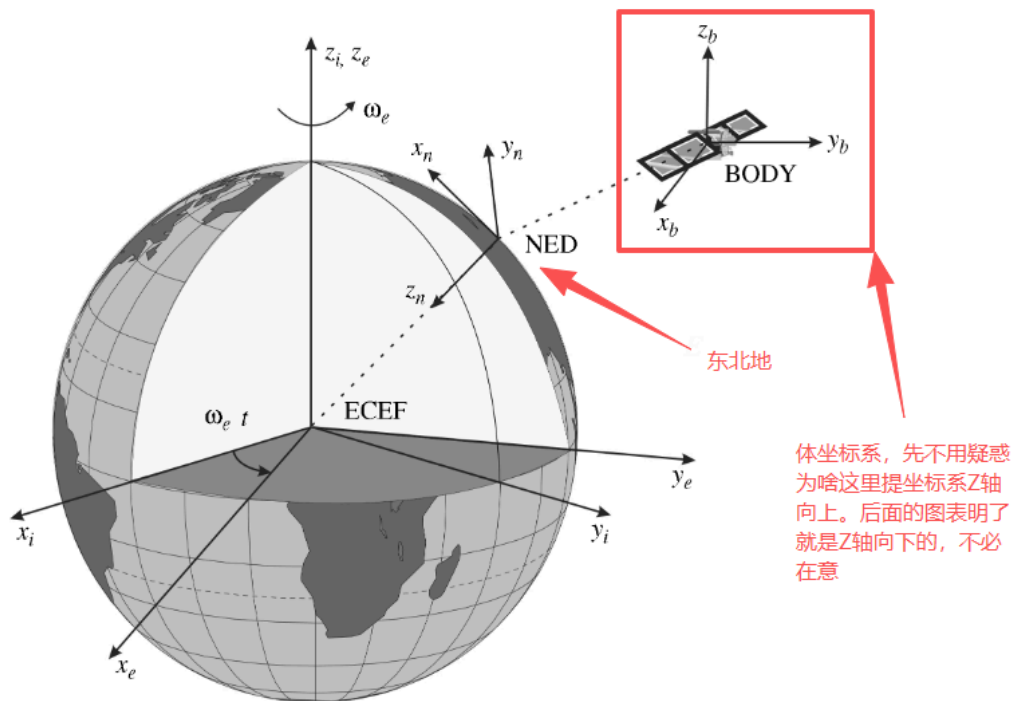


Figure 2.2 The Earth-centered Earth-fixed (ECEF) frame $x_e y_e z_e$ is rotating with angular rate ω_e with respect to an Earth-centered inertial (ECI) frame $x_i y_i z_i$ fixed in space.

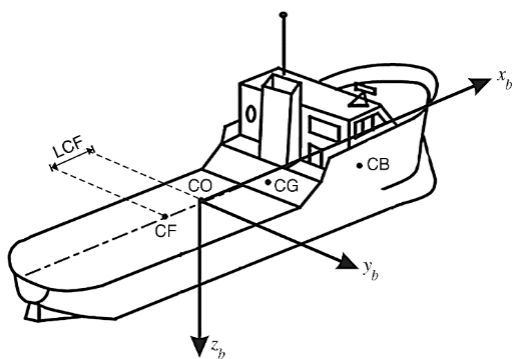


Figure 2.3 Body-fixed reference points.

2.2.1 Euler Angle Transformation

The Euler angles, roll (ϕ), pitch (θ) and yaw (ψ), can now be used to decompose the body-fixed velocity vector $v_{b/n}^b$ in the NED reference frame. Let $R_b^n(\Theta_{nb}) : \mathcal{S}^3 \rightarrow SO(3)$ denote the Euler angle rotation matrix with argument $\Theta_{nb} = [\phi, \theta, \psi]^T$. Hence,

这里是左乘，表明v经过变换从体坐标系到惯性坐标系下的表达

$$v_{b/n}^n = R_b^n(\Theta_{nb}) v_{b/n}^b \quad (2.14)$$

Principal Rotations

The principal rotation matrices (one axis rotations) can be obtained by setting $\lambda = [1, 0, 0]^T$, $\lambda = [0, 1, 0]^T$ and $\lambda = [0, 0, 1]^T$ corresponding to the x, y and z axes, and $\beta = \phi$, $\beta = \theta$ and $\beta = \psi$, respectively, in the formula for $R_{\lambda, \beta}$ given by (2.12). This yields

$$R_{x, \phi} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\phi & -s\phi \\ 0 & s\phi & c\phi \end{bmatrix}, \quad R_{y, \theta} = \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix}, \quad R_{z, \psi} = \begin{bmatrix} c\psi & -s\psi & 0 \\ s\psi & c\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2.15)$$

where $s \cdot = \sin(\cdot)$ and $c \cdot = \cos(\cdot)$.

Linear Velocity Transformation

这里说顺序是zyx 表明的是惯性"坐标系"经过zyx的变换，变成了体"坐标系" 表示的是坐标系的变换

It is customary to describe $R_b^n(\Theta_{nb})$ by three *principal* rotations about the z, y and x axes (zyx convention). Note that the order in which these rotations is carried out is not arbitrary. In guidance, navigation and control applications it is common to use the zyx convention *from {n} to {b}* specified in terms of the Euler angles ϕ , θ and ψ for the rotations. This matrix is denoted $R_b^n(\Theta_{nb}) = R_b^n(\Theta_{nb})^T$. The matrix transpose implies that the same result is obtained by transforming a vector *from {b} to {n}*, that is by reversing the order of the transformation. This rotation sequence is mathematically equivalent to

$$R_b^n(\Theta_{nb}) := R_{z, \psi} R_{y, \theta} R_{x, \phi} \quad (2.16)$$

and the inverse transformation is then written (zyx convention)

无论旋转矩阵还是雅可比矩阵都是反对称矩阵

$$R_b^n(\Theta_{nb})^{-1} = R_n^b(\Theta_{nb}) = R_{x, \phi}^T R_{y, \theta}^T R_{z, \psi}^T \quad (2.17)$$

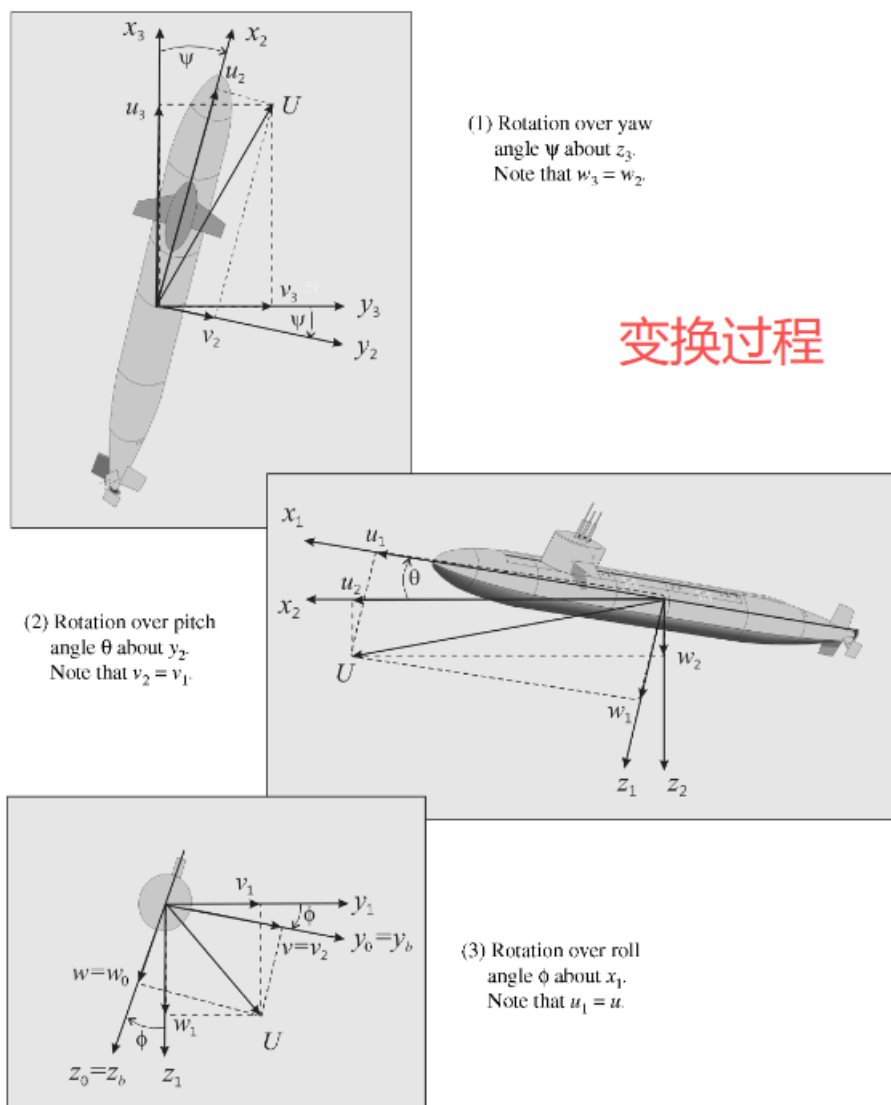
Let $x_3 y_3 z_3$ be the coordinate system obtained by translating the NED coordinate system $x_n y_n z_n$ parallel to itself until its origin coincides with the origin of the body-fixed coordinate system. The coordinate system $x_3 y_3 z_3$ is rotated a yaw angle ψ about the z_3 axis. This yields the coordinate system $x_2 y_2 z_2$. The coordinate system $x_2 y_2 z_2$ is rotated a pitch angle θ about the y_2 axis. This yields the coordinate system $x_1 y_1 z_1$. Finally, the coordinate system $x_1 y_1 z_1$ is rotated a roll angle ϕ about the x_1 axis. This yields the body-fixed coordinate system $x_b y_b z_b$.

Expanding (2.16) yields

解释了变换的过程 惯性坐标系先绕着自己的Z轴转 然后在绕着变换后的Y轴转 再绕着变换后的X轴转 最终得到体坐标系

$$R_b^n(\Theta_{nb}) = \begin{bmatrix} c\psi c\theta & -s\psi c\theta + c\psi s\theta s\phi & s\psi s\theta + c\psi c\theta s\phi \\ s\psi c\theta & c\psi c\theta + s\psi s\theta s\phi & -c\psi s\theta + s\psi c\theta s\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix} \quad (2.18)$$

变换矩阵



变换过程

Figure 2.4 Euler angle rotation sequence (zyx convention). The submarine is rotated from $\{n\}$ to $\{b\}$ by using three principal rotations.

The body-fixed velocity vector $\mathbf{v}_{b/n}^b$ can be expressed in $\{n\}$ as

线速度

$$\dot{\mathbf{p}}_{b/n}^n = \mathbf{R}_b^n(\boldsymbol{\Theta}_{nb}) \mathbf{v}_{b/n}^b \quad (2.20)$$

Important

考虑完线速度，接下来考虑角速度。这里最最最关键的核心，是不要把**“欧拉角速度变化率”**和**“体坐标系下的角速度在惯性坐标系下的表达弄混乱”**

很多同学容易在考虑欧拉角的角速度 $[\dot{\phi}, \dot{\theta}, \dot{\psi}]$ 时，容易把其天然的认为是机器人本体坐标系下的速度 $[p, q, r]$ 乘以上文“转换矩阵”的 J_1 或者 J_1' ，不要这么理解，**我们应该把速度雅可比 J_2 单独拎出来理解和求解。**

在机体坐标系 (Body frame) $\gg \omega_b$ 下是真实角速度向量在 body frame 下的坐标表达。：

$$\omega_b = [p, q, r]$$

也可以表达在惯性坐标系下 $\gg \omega_n$ 是真实角速度向量在 inertial frame 下的坐标表达：

$$\omega_n = R\omega_b$$

注意： ω_b 和 ω_n 本质上是同一个物理角速度，只是坐标系不同。而欧拉角变化率经过如下定义：

欧拉角：

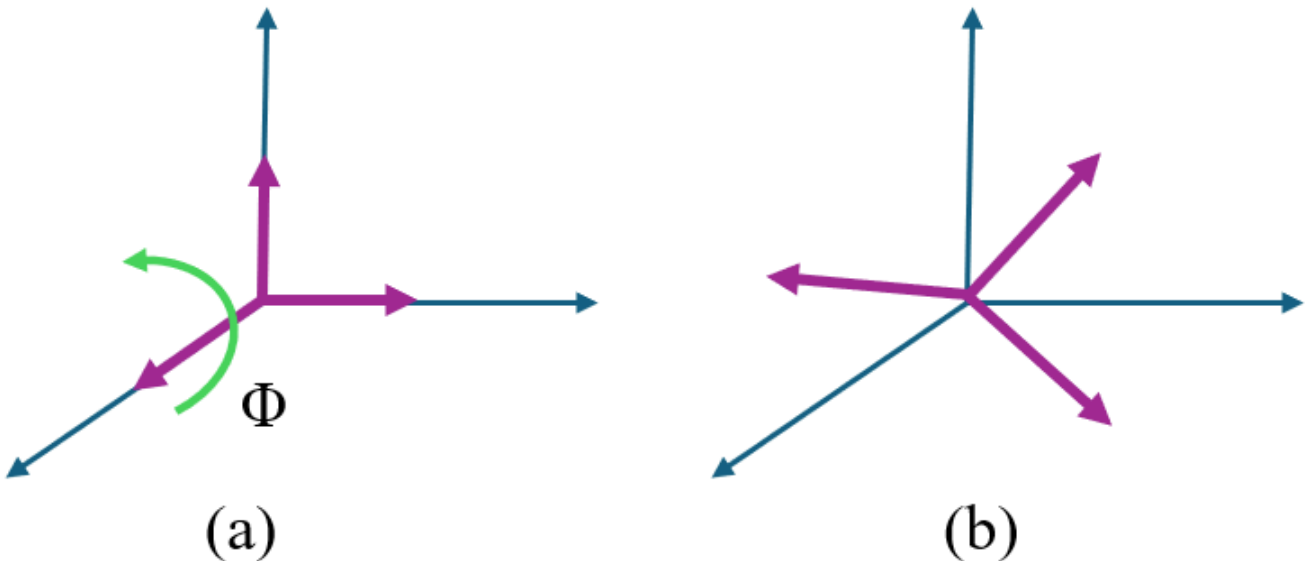
$$(\phi, \theta, \psi)$$

这三个连续旋转步骤的参数。因此：

$$\dot{\Theta} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]$$

仅仅表示欧拉角的参数变化有多快，它不是一个真实的空间角速度向量。

于是在考虑体坐标系下角速度和欧拉角变化率的时候，我们可以重新想象一下。两个坐标系 $\{n\}$ 和 $\{b\}$ 。他们一开始重合在一起如图1.(a)， $\{n\}$ 欧拉角旋转定义是 X→Y→Z。按顺序变换到图(b)。即将欧拉角变化率贡献(或者说投影，最好不要说转换，因为 $[\dot{\phi}, \dot{\theta}, \dot{\psi}]$ 这不是一个向量)到了机器人体坐标系下的体角速度。而若将体角速度乘以上文的转换矩阵，那么我们得到的是体角速度在惯性坐标系下的表达。(为了方便理解，我们完全可以将角速度这边的变化重新认定为一个新的定义的变化)



Angular Velocity Transformation

The body-fixed angular velocity vector $\omega_{b/n}^b = [p, q, r]^T$ and the Euler rate vector $\dot{\Theta}_{nb} = [\dot{\phi}, \dot{\theta}, \dot{\psi}]^T$ are related through a transformation matrix $T_{\Theta}(\Theta_{nb})$ according to

$$\dot{\Theta}_{nb} = T_{\Theta}(\Theta_{nb})\omega_{b/n}^b \quad (2.26)$$

It should be noted that the angular body velocity vector $\omega_{b/n}^b = [p, q, r]^T$ cannot be integrated directly to obtain actual angular coordinates. This is due to the fact that $\int_0^t \omega_{b/n}^b(\tau) d\tau$ does not have any immediate physical interpretation; however, the vector $\Theta_{nb} = [\phi, \theta, \psi]^T$ does represent proper generalized coordinates. The transformation matrix $T_{\Theta}(\Theta_{nb})$ can be derived in several ways, for instance:

先X，后局部Y，后局部Z

$$\omega_{b/n}^b = \begin{bmatrix} \dot{\phi} \\ 0 \\ 0 \end{bmatrix} + R_{x,\phi}^T \begin{bmatrix} 0 \\ \dot{\theta} \\ 0 \end{bmatrix} + R_{x,\phi}^T R_{y,\theta}^T \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix} := T_{\Theta}^{-1}(\Theta_{nb})\dot{\Theta}_{nb} \quad (2.27)$$

This relationship is verified by inspection of Figure 2.4. Expanding (2.27) yields

$$T_{\Theta}^{-1}(\Theta_{nb}) = \begin{bmatrix} 1 & 0 & -s\theta \\ 0 & c\phi & c\theta s\phi \\ 0 & -s\phi & c\theta c\phi \end{bmatrix} \implies T_{\Theta}(\Theta_{nb}) = \begin{bmatrix} 1 & s\phi t\theta & c\phi t\theta \\ 0 & c\phi & -s\phi \\ 0 & s\phi/c\theta & c\phi/c\theta \end{bmatrix} \quad (2.28)$$

where $s \cdot = \sin(\cdot)$, $c \cdot = \cos(\cdot)$ and $t \cdot = \tan(\cdot)$. Expanding (2.26) yields the Euler angle attitude equations in component form:

$$\dot{\phi} = p + q \sin(\phi) \tan(\theta) + r \cos(\phi) \tan(\theta) \quad (2.29)$$

$$\dot{\theta} = q \cos(\phi) - r \sin(\phi) \quad (2.30)$$

$$\dot{\psi} = q \frac{\sin(\phi)}{\cos(\theta)} + r \frac{\cos(\phi)}{\cos(\theta)}, \quad \theta \neq \pm 90^\circ \quad (2.31)$$

□ 因此我们可以得到雅可比矩阵 J_2 为速度旋转雅可比矩阵，其定义如上。

$$J_2 = \begin{bmatrix} 1 & s\phi t\theta & c\phi t\theta \\ 0 & c\phi & -s\phi \\ 0 & s\phi/c\theta & c\phi/c\theta \end{bmatrix}$$

所以很清晰：代码为

```
J1 = Rz*Ry*Rx
J2 = [1, sin(q4(t))*tan(q5(t)), cos(q4(t))*tan(q5(t));
      0, cos(q4(t)), -sin(q4(t));
      0, sin(q4(t))/cos(q5(t)), cos(q4(t))/cos(q5(t))]
```

接下来构造总的雅可比矩阵和雅可比矩阵的逆转：

```
%% 构造总的雅可比矩阵J
J = blkdiag(J1, J2); % 使用 blkdiag 构建 J 矩阵
%% 计算 J 的逆和转置矩阵 A
AT = simplify(inv(J));
A = transpose(AT);
%% 定义惯性矩阵 参考fossen
M_RB = [m, 0, 0, 0, m*zG, -m*yG;
        0, m, 0, -m*zG, 0, m*xG;
        0, 0, m, m*yG, -m*xG, 0;
        0, -m*zG, m*yG, ix, -ixy, -ixz;
        m*zG, 0, -m*xG, -ixy, iy, -iyz;
        -m*yG, m*xG, 0, -ixz, -iyz, iz];
```

接下来定义广义坐标向量和体坐标向量

```

%% 定义速度向量
% 定义广义坐标向量
q_vec = [q1(t); q2(t); q3(t); q4(t); q5(t); q6(t)];
nu = [u(t); v(t); w(t); p(t); q(t); r(t)];
nu_v = v(1:3);
nu_omega = v(4:6);
% v_dot是一个6x1的列向量，包含了所有广义速度的导数。
nu_dot = [diff(u(t),t);
          diff(v(t),t);
          diff(w(t),t);
          diff(p(t),t);
          diff(q(t),t);
          diff(r(t),t)];

```

计算动能

```

%% 计算动能
T = 0.5 * nu.' * M_RB * nu;

```

有了A矩阵，我们在求出相应的 Γ 矩阵

```

%% 求解gama-1
% 初始化三维数组来存储结果
AQVector = sym(zeros(6, 6, 6));
% 遍历矩阵 A 的每个元素并求偏导数
for i = 1:6
    for j = 1:6
        % 获取矩阵 A 的 (i, j) 元素
        A_ij = A(i, j);
        % 遍历广义坐标向量 q_vec 的每个元素，并求偏导数
        for k = 1:6
            AQVector(i, j, k) = diff(A_ij, q_vec(k));
        end
    end
end
gama1 = sym(zeros(6,6)); %建立gama1空矩阵，方便后续存放gama1
for i = 1:6
    for j = 1:6
        % 获取 AQVector 的切片，即 A(i,j) 对所有广义坐标的偏导数组成的列向量。
        % 在 MATLAB 中，这对应于 AQVector(i, j, :)。
        AQ_slice = squeeze(AQVector(i, j, :));
        % x1 的每个元素都是一个向量-矩阵-向量乘积。
        % 注意: w 是 1x6, Transpose(A) 是 6x6, AQ_slice 是6x1。
        % 这三者相乘的结果是一个标量。
        gama1(i, j) = nu.' * j.' * AQ_slice;
    end
end
% disp(gama1)
%% 求解gama-2
DAq1 = diff(A,q1(t));
DAq2 = diff(A,q2(t));
DAq3 = diff(A,q3(t));

```

```

DAq4 = diff(A,q4(t));
DAq5 = diff(A,q5(t));
DAq6 = diff(A,q6(t));

DAq_eta = cat(3,DAq1,DAq2,DAq3,DAq4,DAq5,DAq6); %沿着三维进行拼接，这一句只是用来示意的，不写也行。
仅用于示意的d(A/eta)的形式

gama2_l1 = nu.' * J.' * DAq1;
gama2_l2 = nu.' * J.' * DAq2;
gama2_l3 = nu.' * J.' * DAq3;
gama2_l4 = nu.' * J.' * DAq4;
gama2_l5 = nu.' * J.' * DAq5;
gama2_l6 = nu.' * J.' * DAq6;
gama2 = [gama2_l1;
         gama2_l2;
         gama2_l3;
         gama2_l4;
         gama2_l5;
         gama2_l6;];

%% 求gamma
gama = gama1 - gama2;
disp('gama is:')
disp(gama);

```

$$\frac{d}{dt} \left(\frac{\partial \bar{T}}{\partial \nu} \right) + J^T \mathbf{\Gamma} \left(\frac{\partial \bar{T}}{\partial \nu} \right) - J^T \left(\frac{\partial \bar{T}}{\partial \eta} \right) = \tau$$

$$\mathbf{\Gamma} = \mathbf{\Gamma}_1 - \mathbf{\Gamma}_2$$

$$A = J^{-T}$$

$$\mathbf{\Gamma}_1 = \left[\nu^T J^T \left(\frac{\partial a_{ij}}{\partial \eta} \right) \right]_{6 \times 6}, \quad \mathbf{\Gamma}_2 = \left[\nu^T J^T \left[\frac{\partial A}{\partial \eta_i} \right] \right]_{6 \times 6}$$

我们已经列出了类拉格朗日方程如上，类拉格朗日方程的核心是等式左边的第一项 $\frac{d}{dt} \left(\frac{\partial \bar{T}}{\partial \nu} \right)$ 是主要项，我们可以推导一下：

💡 Tip

我们都知道动能是 $T = \frac{1}{2} m v^2$ ，若是转动则是 $T = \frac{1}{2} I \omega^2$

对于我们的水下机器人的动能，直接就是 $T = \frac{1}{2} m \nu^2$

对速度求导则有 $\frac{dT}{d\nu} = m\nu = \text{动量}$

而动量对时间求导则是力\力矩，那么对于我们的水下机器人而言，为 $\frac{d}{dt} \frac{dT}{d\nu} = M_{RB} \dot{\nu} = M \dot{\nu} + C_1(\nu) \nu = \tau$

那么如何求 M 呢？很简单，我们直接再对 $\dot{\nu}$ 求一次倒，那么 $M \dot{\nu} + C_1(\nu) \nu$ 就只剩下 M 了（注：直到目前位置我们都没有考虑重力）

对应部分代码如下：

% H1 的计算

```
H1_temp = expand(diff(diff(T, u(t)), t));
```

```
H1 = simplify(jacobian(H1_temp, nu_dot));
```

H1_rest = simplify(H1_temp - H1*nu_dot); %H1_rest表示 拉格朗日法动力学建模求解的中间项， 其实也就是惯性力和科氏力矩阵C的一部分

我们很容易模仿，得到如下

% H2 的计算

```
H2_temp = expand(diff(diff(T, v(t)), t));
```

```
H2 = simplify(jacobian(H2_temp, nu_dot));
```

```
H2_rest = simplify(H2_temp - H2*nu_dot);
```

% H3 的计算

```
H3_temp = expand(diff(diff(T, w(t)), t));
```

```
H3 = simplify(jacobian(H3_temp, nu_dot));
```

```
H3_rest = simplify(H3_temp - H3*nu_dot);
```

% H4 的计算

```
H4_temp = expand(diff(diff(T, p(t)), t));
```

```
H4 = simplify(jacobian(H4_temp, nu_dot));
```

```
H4_rest = simplify(H4_temp - H4*nu_dot);
```

% H5 的计算

```
H5_temp = expand(diff(diff(T, q(t)), t));
```

```
H5 = simplify(jacobian(H5_temp, nu_dot));
```

```
H5_rest = simplify(H5_temp - H5*nu_dot);
```

% H6 的计算

```
H6_temp = expand(diff(diff(T, r(t)), t));
```

```
H6 = simplify(jacobian(H6_temp, nu_dot));
```

```
H6_rest = simplify(H6_temp - H6*nu_dot);
```

```
H = [H1;H2;H3;H4;H5;H6];
```

```
disp('H:')
```

```
disp(H)
```

```
H_rest_total = [H1_rest;
```

```
                H2_rest;
```

```
                H3_rest;
```

```
                H4_rest;
```

```
                H5_rest;
```

```
                H6_rest;];
```

```
test_sum = H - H.';
```

```
disp('test_sum: ')
```

```
disp(test_sum)
```

接下来需要求整体的偏置项，即总的惯性力和离心力矩阵 C 。上文我们已经推导了 M 。我们来推导一下 C

💡 Tip

类拉格朗日方程等式左侧的第二第三项分别是“坐标变换带来的附加科氏力项”以及“位置相关力项”，这两项分别都可以看作是修正项，它们都是科氏力和离心力矩阵 C 的一部分。而我们再上文已经求出了 C 的一部分 $C_1(\nu)\nu$

注意：是 $C_1(\nu)\nu$ ，对应代码中的**H_rest_total**

那么 $J^T \mathbf{T} \left(\frac{\partial \bar{T}}{\partial \nu} \right)$ 构成第二分量 $C_2(\nu)\nu$ ， $-J^T \left(\frac{\partial \bar{T}}{\partial \eta} \right)$ 构成第三分量 $C_3(\nu)\nu$

```
%% 求矩阵C(v,v_dot)
% 计算动能对类速度向量偏导
dTdvv = jacobian(T,nu);
JTgama = J.'*gama;
dTdeta = jacobian(T,q_vec);
% 第一C_nu分量
C_RB_1 = simplify(H_rest_total);
% 第二C_nu分量
C_RB_2 = simplify(JTgama * dTdvv. ');
% 第三C_nu分量
C_RB_3 = simplify(J.' * dTdeta. ');

C_nu = simplify(C_RB_1 + C_RB_2 + C_RB_3);
C = sixdof_C_decouple(C_nu); %
C = simplify(C);
disp(C)
%验证部分，不等于0，说明用简单的jacobian偏微分起不到等同的效果
C_test = simplify(jacobian(C_nu,nu));
C_sum_test = simplify(C_test - C*nu);
disp(C_sum_test);
```

sixdof_C_decouple是将复杂的向量才分开成两个元素相乘的函数。将 $C(\nu)\nu$ 拆分成 $C(\nu)$ 和 ν 。

这样我们就可以得到 $C(\nu)$

重浮力与水阻项(Recovery & Damping)

接下来我们看重力部分。

$$g(\eta) = \begin{bmatrix} (W - B) s\theta \\ - (W - B) c\theta s\phi \\ - (W - B) c\theta c\phi \\ - (y_G W - y_B B) c\theta c\phi + (z_G W - z_B B) c\theta s\phi \\ (z_G W - z_B B) s\theta + (x_G W - x_B B) c\theta c\phi \\ - (x_G W - x_B B) c\theta s\phi - (y_G W - y_B B) s\theta \end{bmatrix} \quad (2.168)$$

重力部分其实就是重心和浮心的综合影响。包括了力和力矩

```
%% 重力项
Position_G = [xG,yG,zG];
Position_B = [xB,yB,zB];
```

```
G = gravity_6dof(q4(t),q5(t),w,B,Position_G,Position_B);

function G = gravity_6dof(theta,phi,w,B,Position_G,Position_B)
delta1 = w-B;
xG = Position_G(1);
yG = Position_G(2);
zG = Position_G(3);

xB = Position_B(1);
yB = Position_B(2);
zB = Position_B(3);
G(1) = delta1*sin(theta);
G(2) = -delta1*cos(theta)*sin(phi);
G(3) = -delta1*cos(theta)*sin(theta);
G(4) = -(yG*w - yB*B)*cos(theta)*cos(phi) + (zG*w - zB*B)*cos(theta)*sin(phi);
G(5) = (zG*w - zB*B)*sin(theta) + (xG*w - xB*B)*cos(theta)*cos(phi);
G(6) = -(xG*w - xB*B)*cos(theta)*sin(phi)-(yG*w - yB*B)*sin(theta);
end
```

再看水阻部分。主要是通过水动力参数识别得到，直接代入真实值

```
%% 水阻力项目，通过水动力参数识别获得。直接写数值
DL = diag([12,23,157,0.016,0.78,0.78]); %线性阻尼
DV = diag([100,240,100,0.87,2.16,2.16]); %二次阻尼
D = (DL + DV)*nu; %体坐标系
D_eta = ((J.')\D)./J; %惯性坐标系
```

环境力建模(Disturbance)

💡 Tip

主要在算法里使用

环境力建模也是直接在求解的过程中带参数值。

```
function tau_env = disturbance_6dof(t, nu_k, J_inv_k_full, Z_depth, DL_env)
%% 环境扰动参数
% 洋流速度，惯性坐标系下
NU_C_INERTIAL = [1; 0.8; 0.2; 0; 0; 0];
% 波浪力参数
A_WAVE = 5; % 波浪力幅值 N
OMEGA_WAVE = 0.5; % 波浪频率 rad/s
K_WAVE_DECAY = 1.0; % 深度衰减系数
% 随机噪声参数
K_NOISE = diag([2, 2, 1, 0.2, 0.2, 0.2]);
%% 初始化
tau_env = zeros(6, 1); % 体坐标系下的6DOF环境扰动
%% 1. 洋流扰动
% 惯性系洋流速度转换到体坐标系
nu_c_body = J_inv_k_full(1:6, 1:6) * NU_C_INERTIAL;
% 相对速度
nu_r = nu_k - nu_c_body;
```

```

% 洋流造成的阻力
tau_current = -DL_env * nu_r;
tau_env = tau_env + tau_current;
%% 2. 波浪扰动
% 深度越深，波浪影响越小
wave_decay = exp(-K_WAVE_DECAY * abs(Z_depth));
wave_force_x = A_WAVE * sin(OMEGA_WAVE * t) * wave_decay;
wave_force_y = A_WAVE * cos(OMEGA_WAVE * t) * wave_decay;
tau_wave_body = [wave_force_x; wave_force_y; 0; 0; 0; 0];
tau_env = tau_env + tau_wave_body;
%% 3. 随机噪声
tau_noise = K_NOISE * randn(6, 1);
tau_env = tau_env + tau_noise;
end

```

推力分配(Distribution)

ROV-Mode

```

function A_thruster = ROV_Distribution(gama,beta,L,p,R)
%% 推力分配
%注意推进器的分布 后面从左往右12 前面从左往右34 然后中间从左往右56 最后一个7
param1 = cos(gama);
param2 = sin(gama);
param3 = cos(beta);
A_thruster = [-param1, -param1, param1, param1, 0, 0, 0;
              -param2, param2, -param2, param2, 0, 0, 0;
              0, 0, 0, 0, 1, 1, 1;
              -L*param2*param3, L*param2*param3, -L*param2*param3, L*param2*param3, R, -R,
0;
              L*param1*param3, L*param1*param3, -L*param1*param3, -L*param1*param3, 0, 0,
R;
              -p, p, p, -p, 0, 0, 0]; % 6x7 矩阵 7个推进器
end

```

QUAD-Mode

```

function A_thruster = QUAD_Distribution(gama,beta,L,R,k)
%% 推力分配
%注意推进器的分布 后面从左往右12 前面从左往右34 然后中间从左往右56 最后一个7
param1 = cos(gama);
param2 = sin(gama);
param3 = cos(beta);
param4 = sin(beta);
A_thruster = [-param2*param3, -param2*param3, param2*param3, param2*param3, 0, 0, 0;
              param1*param3, -param1*param3, param1*param3, -param1*param3, 0, 0, 0;
              param4, param4, param4, param4, 1, 1, 1;
              L*param4, -L*param4, L*param4, -L*param4, R, -R, 0;
              L*param4, L*param4, -L*param4, -L*param4, 0, 0, R;
              k*param3*param1, -k*param3*param1, -k*param3*param1, k*param3*param1, 0, 0,
0]; % 6x7 矩阵 7个推进器
end

```

Wheel-Mode

```
function A_thruster = wheel_Distribution(k_p,R)
%% 推力分配
%注意推进器的分布 后面从左往右12 前面从左往右34 然后中间从左往右56 最后一个7
A_thruster = [k_p, k_p, k_p, k_p, 0, 0, 0;
              0, 0, 0, 0, 0, 0, 0;
              0, 0, 0, 0, 1, 1, 1;
              0, 0, 0, 0, R, -R, 0;
              0, 0, 0, 0, 0, 0, R;
              k_p, -k_p, k_p, -k_p, 0, 0, 0]; % 6x7 矩阵 7个推进器

end
```

统一的动力学建模部分(七自由度带载具)——抵近观测臂(UVOS)

同理上部分。但是这时候我们多了一些转换矩阵的操作，和新增了载具的自由度。

```
%% 7自由度带载具类拉格朗日建模
clear;
clc;
close all;
syms t real
syms m m1 m2 L0 Ls g real
syms ix iy iz ixy ixz iyz i_x1 i_y1 i_z1 i_xy1 i_xz1 i_yz1 real
syms xG yG zG xG1 yG1 zG1 rMx rMy rMz real %机器人重心 机械臂重心 机械臂基座位置
syms xB yB zB %机器人浮心
syms W B W_o% 机器人重浮力项 机械臂重力项
syms q1(t) q2(t) q3(t) q4(t) q5(t) q6(t) a_d(t)
syms u(t) v(t) w(t) p(t) q(t) r(t)
syms u_d(t) v_d(t) w_d(t) p_d(t) q_d(t) r_d(t)
syms phi(t) theta(t) ksi(t)
```

📌 Important

多了自由度 $a_d(t)$ 这是观测臂的平动自由度

```
%% 定义旋转矩阵
Rx = [1, 0, 0;
      0, cos(q4(t)), -sin(q4(t));
      0, sin(q4(t)), cos(q4(t))];
Ry = [cos(q5(t)), 0, sin(q5(t));
      0, 1, 0;
      -sin(q5(t)), 0, cos(q5(t))];
Rz = [cos(q6(t)), -sin(q6(t)), 0;
      sin(q6(t)), cos(q6(t)), 0;
      0, 0, 1];
%% 计算组合旋转矩阵
J1 = Rz*Ry*Rx;
J2 = [1, sin(q4(t))*tan(q5(t)), cos(q4(t))*tan(q5(t));
      0, cos(q4(t)), -sin(q4(t));
      0, sin(q4(t))/cos(q5(t)), cos(q4(t))/cos(q5(t))];
%% 构造雅可比矩阵J
```

```

J_sub_matrix1 = J1;
J_sub_matrix2 = J2;
J_sub_matrix3 = eye(1);
J_m = blkdiag(J_sub_matrix1, J_sub_matrix2); % 使用 blkdiag 构建 B1 矩阵
J = blkdiag(J_m, J_sub_matrix3);
%% 定义惯性矩阵
M_RB = [m, 0, 0, 0, m*zG, -m*yG;
         0, m, 0, -m*zG, 0, m*xG;
         0, 0, m, m*yG, -m*xG, 0;
         0, -m*zG, m*yG, ix, -ixy, -ixz;
         m*zG, 0, -m*xG, -ixy, iy, -iyz;
         -m*yG, m*xG, 0, -ixz, -iyz, iz];
M_1 = [m1, 0, 0, 0, m1*zG1, -m1*yG1;
        0, m1, 0, -m1*zG1, 0, m1*xG1;
        0, 0, m1, m1*yG1, -m1*xG1, 0;
        0, -m1*zG1, m1*yG1, i_x1, -i_xy1, -i_xz1;
        m1*zG1, 0, -m1*xG1, -i_xy1, i_y1, -i_yz1;
        -m1*yG1, m1*xG1, 0, -i_xz1, -i_yz1, i_z1];
%% 定义速度向量
% 定义广义坐标向量
q_vec = [q1(t); q2(t); q3(t); q4(t); q5(t); q6(t); a_d(t)];
nu = [u(t); v(t); w(t); p(t); q(t); r(t)];
vv = nu(1:3);
omegav = nu(4:6);
nu_total = [u(t); v(t); w(t); p(t); q(t); r(t); diff(a_d(t),t)];
% nu_dot_total是一个7x1的列向量，包含了所有广义速度的导数。
nu_dot_total = [diff(u(t),t);
                diff(v(t),t);
                diff(w(t),t);
                diff(p(t),t);
                diff(q(t),t);
                diff(r(t),t);
                diff(a_d(t),t,2)];

```

Important

注意：这里是** nu_total 和 nu_dot_total**

关键部分。观测臂的坐标系定义和速度转换

Important

```

%% 坐标系定义与转换
% 这部分代码定义了机械臂和机器人本体之间的坐标系关系。
rM = [rMx; rMy; rMz]; % rM表示机械臂基座（即机械臂与机器人本体的连接点）相对于机器人本体原点的位置向量。
RV0 = [0, 1, 0; 1, 0, 0; 0, 0, -1]; % 0->V {0}机械臂的基座系哦
R0V = RV0.';
rL1 = RV0 * [0; L1 + diff(a_d(t),t); 0]; %L1是伸缩长度基量
%% 速度转换
v1 = vv + cross(omegav, rM + rL1);
omega1 = omegav;
v11 = R0V * v1; %v->1
omega11 = R0V * omega1;
v1 = [v11; omega11];

```

坐标转换和速度定义详细看推力过程。

接下来差不多，计算动能之后步步求解

```

%% 计算动能
T = 0.5 * nu.' * M_RB * nu + 0.5 * v1.' * M_1 * v1;
%% 求解gama-1
% 初始化三维数组来存储结果
AQvector = sym(zeros(7, 7, 7));
% 遍历矩阵 A 的每个元素并求偏导数
for i = 1:7
    for j = 1:7
        % 获取矩阵 A 的 (i, j) 元素
        A_ij = A(i, j);
        % 遍历广义坐标向量 q_vec 的每个元素，并求偏导数
        for k = 1:7
            AQvector(i, j, k) = diff(A_ij, q_vec(k));
        end
    end
end
gama1 = sym(zeros(7,7));
for i = 1:7
    for j = 1:7
        % 获取 AQvector 的切片，即 A(i,j) 对所有广义坐标的偏导数组成的列向量。
        % 在 MATLAB 中，这对应于 AQvector(i, j, :)。
        AQ_slice = squeeze(AQvector(i, j, :));

        % x1 的每个元素都是一个向量-矩阵-向量乘积。
        % 注意: nu_total是 1x7, Transpose(J) 是 7x7, AQ_slice 是 7x1。
        % 这三者相乘的结果是一个标量。
        gama1(i, j) = nu_total.' * J.' * AQ_slice;
    end
end
% disp(gama1)
%% 求解gama-2

DAq1 = diff(A, q1(t));
DAq2 = diff(A, q2(t));

```

```

DAq3 = diff(A,q3(t));
DAq4 = diff(A,q4(t));
DAq5 = diff(A,q5(t));
DAq6 = diff(A,q6(t));
DAq7 = diff(A,a_d(t));

DAq_eta = cat(3,DAq1,DAq2,DAq3,DAq4,DAq5,DAq6,DAq7);

gama2_l1 = nu_total.' * J.' * DAq1;
gama2_l2 = nu_total.' * J.' * DAq2;
gama2_l3 = nu_total.' * J.' * DAq3;
gama2_l4 = nu_total.' * J.' * DAq4;
gama2_l5 = nu_total.' * J.' * DAq5;
gama2_l6 = nu_total.' * J.' * DAq6;
gama2_l7 = nu_total.' * J.' * DAq7;

gama2 = [gama2_l1;
        gama2_l2;
        gama2_l3;
        gama2_l4;
        gama2_l5;
        gama2_l6;
        gama2_l7;];

%% 求gamma
gama = gama1 - gama2;
disp('gama is:')
disp(gama);

%%
% H1 的计算
H1_temp = expand(diff(diff(T, u(t)), t));

H1 = simplify(jacobian(H1_temp,nu_dot_total));

H1_rest = simplify(H1_temp - H1*nu_dot_total);%H1_rest表示 拉格朗日法动力学建模求解的中间项， 其实
也就是惯性力和科氏力矩阵C的一部分

% H2 的计算
H2_temp = expand(diff(diff(T, v(t)), t));

H2 = simplify(jacobian(H2_temp,nu_dot_total));

H2_rest = simplify(H2_temp - H2*nu_dot_total);

% H3 的计算
H3_temp = expand(diff(diff(T, w(t)), t));

H3 = simplify(jacobian(H3_temp,nu_dot_total));

H3_rest = simplify(H3_temp - H3*nu_dot_total);

% H4 的计算
H4_temp = expand(diff(diff(T, p(t)), t));

```



```

H4 = simplify(jacobian(H4_temp,nu_dot_total));

H4_rest = simplify(H4_temp - H4*nu_dot_total);

% H5 的计算
H5_temp = expand(diff(diff(T, q(t)), t));

H5 = simplify(jacobian(H5_temp,nu_dot_total));

H5_rest = simplify(H5_temp - H5*nu_dot_total);

% H6 的计算
H6_temp = expand(diff(diff(T, r(t)), t));

H6 = simplify(jacobian(H6_temp,nu_dot_total));

H6_rest = simplify(H6_temp - H6*nu_dot_total);

% H7 的计算
H7_temp = expand(diff(diff(T, diff(a_d(t),t)), t));

H7 = simplify(jacobian(H7_temp,nu_dot_total));

H7_rest = simplify(H7_temp - H7*nu_dot_total);

H = [H1;H2;H3;H4;H5;H6;H7];

disp('H:')
disp(H)
H_rest_total = [H1_rest;
                H2_rest;
                H3_rest;
                H4_rest;
                H5_rest;
                H6_rest;
                H7_rest];

test_sum = H - H.';
disp('test_sum: ')
disp(test_sum)

%% 求矩阵C(v,v_dot)
% 计算动能对类速度向量偏导
dTdvv = jacobian(T,nu_total);
JTgama = J.'*gama;
dTdeta = jacobian(T,q_vec);
% 第一C_nu分量
C_RB_1 = simplify(H_rest_total);
% 第二C_nu分量
C_RB_2 = simplify(JTgama * dTdvv.);
% 第三C_nu分量
C_RB_3 = simplify(J.' * dTdeta.) ;

```

```

C_nu = simplify(C_RB_1 + C_RB_2 + C_RB_3);
C = sevendof_C_decouple(C_nu); %
C = simplify(C);
disp(C)
%验证部分，不等于0，说明用简单的jacobian偏微分起不到等同的效果
C_test = simplify(jacobian(C_nu,nu_total));
C_sum_test = simplify(C_test - C*nu_total);
disp(C_sum_test);

```

重浮力与水阻项(Recovery & Damping)

Note

重力和阻力项多了观测臂部分

先看重力部分

```

%% 重力
Position_G = [xG,yG,zG];
Position_B = [xB,yB,zB];
L1 = Ls + diff(a_d(t),t);
G = gravity_with_observer(q4(t),q5(t),W,W_o,B,Position_G,Position_B,L0,L1);

```

**** gravity_with_observer****代码，无需多言，Fossen书籍给定，也很好推导。主要就是重心和浮心的差距导致的力和力矩。值得注意的是带上观测臂后，多出来的一项怎么计算。已经在推导中给出。下面直接看代码：

```

function G = gravity_with_observer(theta,phi,W,W_o,B,Position_G,Position_B,L0,L1)
delta1 = W-B;
xG = Position_G(1);
yG = Position_G(2);
zG = Position_G(3);

xB = Position_B(1);
yB = Position_B(2);
zB = Position_B(3);
G(1) = delta1*sin(theta);
G(2) = -delta1*cos(theta)*sin(phi);
G(3) = -delta1*cos(theta)*sin(theta);
G(4) = -(yG*W - yB*B)*cos(theta)*cos(phi) + (zG*W - zB*B)*cos(theta)*sin(phi);
G(5) = (zG*W - zB*B)*sin(theta) + (xG*W - xB*B)*cos(theta)*cos(phi);
G(6) = -(xG*W - xB*B)*cos(theta)*sin(phi)-(yG*W - yB*B)*sin(theta);
G(7) = W_o*(-L0*cos(theta)*sin(phi) - L0*sin(theta) -L1*cos(theta)*cos(phi) +
L1*cos(theta)*sin(phi));
end

```

最后是水阻部分，也很简单，一般是水动力参数识别后直接带入。

```

%% 水阻力项目，通过水动力参数识别获得。直接写数值
DL = diag([12,23,157,0.016,0.78,0.78,1e-7]); %线性阻尼
DV = diag([100,240,100,0.87,2.16,2.16,0]); %二次阻尼
D = (DL + DV)*nu_total; %体坐标系
D_eta = ((J.')\D)./J %惯性坐标系

```

环境力建模(Disturbance)

💡 Tip

主要在算法里使用

搭载了观测臂的机器人需要考虑环境力(Ocean Current) 还有 观测时的接触力的影响

Ocean && Current

```

function tau_env = disturbance_7dof(t, nu_k, J_inv_k_full, Z_depth, DL_env)
    %% 环境扰动参数
    % 洋流速度，惯性坐标系下
    NU_C_INERTIAL = [1; 0.8; 0.2; 0; 0; 0];
    % 波浪力参数
    A_WAVE = 5; % 波浪力幅值 N
    OMEGA_WAVE = 0.5; % 波浪频率 rad/s
    K_WAVE_DECAY = 1.0; % 深度衰减系数
    % 随机噪声参数
    K_NOISE = diag([2, 2, 1, 0.2, 0.2, 0.2]);
    %% 初始化
    tau_env = zeros(6, 1); % 体坐标系下的6DOF环境扰动
    %% 1. 洋流扰动
    % 惯性系洋流速度转换到体坐标系
    nu_c_body = J_inv_k_full(1:6, 1:6) * NU_C_INERTIAL;
    % 相对速度
    nu_r = nu_k - nu_c_body;
    % 洋流造成的阻力
    tau_current = -DL_env * nu_r;
    tau_env = tau_env + tau_current;
    %% 2. 波浪扰动
    % 深度越深，波浪影响越小
    wave_decay = exp(-K_WAVE_DECAY * abs(Z_depth));
    wave_force_x = A_WAVE * sin(OMEGA_WAVE * t) * wave_decay;
    wave_force_y = A_WAVE * cos(OMEGA_WAVE * t) * wave_decay;
    tau_wave_body = [wave_force_x; wave_force_y; 0; 0; 0; 0];
    tau_env = tau_env + tau_wave_body;
    %% 3. 随机噪声
    tau_noise = K_NOISE * randn(6, 1);
    tau_env = tau_env + tau_noise;

end

```

Contact

💡 Tip

对于接触力，由于搭载的观测臂具备一些储能结构，因此在抵近观测的时候，作用力相当于结构物给的反作用力，其中有一部分被储能元件吸收。可以考虑**Kelvin-Voigt 接触模型**

```
function tau_contact_7 = contact_force_observer(eta_k, nu_k, alpha, alpha_dot)

%% =====
% 7DOF 抵近观测接触力模型
% 前6维: 艇体 [X Y Z K M N]
% 第7维: 伸缩观测臂 alpha
% =====

tau_contact_7 = zeros(7, 1);
%% 1. 参数定义

% 观测臂参数
L0 = 0.5; % 观测臂固定长度 m
z_offset = 0.0; % 末端相对艇体质心的Z向偏置 m
% 圆柱结构物参数
cyl_center = [3; 5]; % 圆柱中心轴在惯性系XY平面的坐标
R_cyl = 0.5; % 圆柱半径 m, 可根据实际改
H_cyl = 10; % 圆柱高度 m
z_bottom = H_cyl; % 圆柱底部z坐标
z_top = 0;
% 接触模型参数
k_contact = 500; % 接触刚度 N/m
c_contact = 50; % 接触阻尼 N/(m/s)
mu = 0.3; % 库伦摩擦系数
eps_v = 1e-6; % 防止除零

%% 2. 状态量读取

pos_i = eta_k(1:3);
phi = eta_k(4);
theta = eta_k(5);
psi = eta_k(6);

v_body = nu_k(1:3);
omega_body = nu_k(4:6);
%% 3. body -> inertial 旋转矩阵
R_bi = eul2rotm_zyx(phi, theta, psi);
%% 4. 观测臂末端位置
% 伸缩方向默认沿体坐标系x轴
e_alpha_b = [1; 0; 0];
% 末端在体坐标系下的位置
r_tip_b = [L0 + alpha;
           0;
           z_offset];
% 末端在惯性坐标系下的位置
p_tip_i = pos_i + R_bi * r_tip_b;
```

```

%% 5. 圆柱高度判断
    %若是末端的高度不在被测物高度范围内，直接退出
    if p_tip_i(3) < z_bottom || p_tip_i(3) > z_top
        return;
    end
%% 6. 圆柱侧表面接触判断
    % 末端相对圆柱中心轴的水平向量
    rho_vec = [p_tip_i(1) - cyl_center(1);
               p_tip_i(2) - cyl_center(2)];
    rho = norm(rho_vec);
    % 防止刚好在中心轴导致除零
    if rho < 1e-6
        n_i = [1; 0; 0];
    else
        % 圆柱径向单位法向量，惯性系下
        n_i = [rho_vec(1) / rho;
               rho_vec(2) / rho;
               0];
    end
    % 到圆柱侧表面的法向距离
    % d > 0: 在圆柱外部，未接触
    % d = 0: 刚好接触
    % d < 0: 压入圆柱内部
    d = rho - R_cyl;
    % 接触压缩量
    delta = max(0, -d);
    if delta <= 0
        return;
    end

%% 7. 观测臂末端速度

    % 伸缩速度，体坐标系下
    v_alpha_b = alpha_dot * e_alpha_b;
    % 末端速度，体坐标系下
    v_tip_b = v_body + cross(omega_body, r_tip_b) + v_alpha_b;
    % 转换到惯性系
    v_tip_i = R_bi * v_tip_b;

%% 8. 法向接触力 Kelvin-Voigt 模型

    % 法向速度，标量
    v_n = dot(v_tip_i, n_i);
    % 法向力大小
    F_n_mag = k_contact * delta - c_contact * v_n;
    % 结构物只能提供反作用力，不能吸引机器人
    F_n_mag = max(0, F_n_mag);
    % 法向接触力，惯性系
    F_n_i = F_n_mag * n_i;

%% 9. 切向摩擦力

    % 切向速度

```

```

v_t_i = v_tip_i - v_n * n_i;
% 库伦摩擦力
F_t_i = -mu * F_n_mag * v_t_i / (norm(v_t_i) + eps_v);

%% 10. 总接触力
F_contact_i = F_n_i + F_t_i;

%% 11. 转换到体坐标系
F_contact_b = R_bi' * F_contact_i;
% 接触力矩，作用点是观测臂末端
M_contact_b = cross(r_tip_b, F_contact_b);

%% 12. 组成7DOF广义接触力

tau_contact_7(1:3) = F_contact_b;
tau_contact_7(4:6) = M_contact_b;
% 第7维：沿伸缩方向的接触广义力
tau_contact_7(7) = dot(F_contact_b, e_alpha_b);
end
function R = eul2rotm_zyx(phi, theta, psi)

RZ = [cos(psi), -sin(psi), 0;
      sin(psi),  cos(psi), 0;
      0,         0,       1];

Ry = [cos(theta), 0, sin(theta);
      0,          1, 0;
      -sin(theta), 0, cos(theta)];

Rx = [1, 0, 0;
      0, cos(phi), -sin(phi);
      0, sin(phi),  cos(phi)];

R = RZ * Ry * Rx;
end

```

推力分配

ROV-With-Observer-Distribution

```

function A_thruster = ROV_with_observer_dist(gama,beta,L,p,R)
%% 推力分配
%注意推进器的分布 后面从左往右12 前面从左往右34 然后中间从左往右56 最后一个7
param1 = cos(gama);
param2 = sin(gama);
param3 = cos(beta);
A_thruster = [-param1, -param1, param1, param1, 0, 0, 0, 0;
              -param2, param2, -param2, param2, 0, 0, 0, 0;
              0, 0, 0, 0, 1, 1, 1, 0;
              -L*param2*param3, L*param2*param3, -L*param2*param3, L*param2*param3, R, -R,
0, 0;
              L*param1*param3, L*param1*param3, -L*param1*param3, -L*param1*param3, 0, 0,
R, 0;

```

```

        -p, p, p, -p, 0, 0, 0, 0;
        0, 0, 0, 0, 0, 0, 0, 1]; % 7x8 矩阵 7个推进器 1个主动推进
end

```

统一的动力学建模部分(七自由度带载具)——采集器(UVCS)

```

%% UVCS的控制模型推导， 转换坐标轴后机械臂是绕着x轴转动的
clear;
clc;
close all;
syms t real
syms m m1 m2 L0 L1 g real
syms ix iy iz ixy ixz iyz i_x1 i_y1 i_z1 i_xy1 i_xz1 i_yz1 real
syms xG yG zG xG1 yG1 zG1 rMx rMy rMz real
syms xB yB zB %机器人浮心
syms W B W_c% 机器人重浮力项 机械臂重力项
syms q1(t) q2(t) q3(t) q4(t) q5(t) q6(t) alpha1(t)
syms u(t) v(t) w(t) p(t) q(t) r(t)
syms u_d(t) v_d(t) w_d(t) p_d(t) q_d(t) r_d(t)
syms phi(t) theta(t) ksi(t)
%% 定义旋转矩阵
Rx = [1, 0, 0;
      0, cos(q4(t)), -sin(q4(t));
      0, sin(q4(t)), cos(q4(t))];
Ry = [cos(q5(t)), 0, sin(q5(t));
      0, 1, 0;
      -sin(q5(t)), 0, cos(q5(t))];
Rz = [cos(q6(t)), -sin(q6(t)), 0;
      sin(q6(t)), cos(q6(t)), 0;
      0, 0, 1];
%% 计算组合旋转矩阵
J1 = Rz*Ry*Rx;
J2 = [1, sin(q4(t))*tan(q5(t)), cos(q4(t))*tan(q5(t));
      0, cos(q4(t)), -sin(q4(t));
      0, sin(q4(t))/cos(q5(t)), cos(q4(t))/cos(q5(t))];
%% 构造雅可比矩阵J
J_sub_matrix1 = J1;
J_sub_matrix2 = J2;
J_sub_matrix3 = eye(1);
J_m = blkdiag(J_sub_matrix1, J_sub_matrix2); % 使用 blkdiag 构建 B1 矩阵
J = blkdiag(J_m, J_sub_matrix3);
%% 计算 B 的逆和转置矩阵 A
AT = simplify(inv(J));
A = transpose(AT);
%% 定义惯性矩阵
M_RB = [m, 0, 0, 0, m*zG, -m*yG;
        0, m, 0, -m*zG, 0, m*xG;
        0, 0, m, m*yG, -m*xG, 0;
        0, -m*zG, m*yG, ix, -ixy, -ixz;
        m*zG, 0, -m*xG, -ixy, iy, -iyz;
        -m*yG, m*xG, 0, -ixz, -iyz, iz];

```

```

M_1 = [m1, 0, 0, 0, m1*zG1, -m1*yG1;
        0, m1, 0, -m1*zG1, 0, m1*xG1;
        0, 0, m1, m1*yG1, -m1*xG1, 0;
        0, -m1*zG1, m1*yG1, i_x1, -i_xy1, -i_xz1;
        m1*zG1, 0, -m1*xG1, -i_xy1, i_y1, -i_yz1;
        -m1*yG1, m1*xG1, 0, -i_xz1, -i_yz1, i_z1];

%% 定义速度向量
% 定义广义坐标向量
q_vec = [q1(t); q2(t); q3(t); q4(t); q5(t); q6(t); alpha1(t)];
nu = [u(t); v(t); w(t); p(t); q(t); r(t)];
vv = nu(1:3);
omegav = nu(4:6);

nu_total = [u(t); v(t); w(t); p(t); q(t); r(t); diff(alpha1(t),t)];
% vv_dot_total是一个7x1的列向量，包含了所有广义速度的导数。
nu_dot_total = [diff(u(t),t);
                diff(v(t),t);
                diff(w(t),t);
                diff(p(t),t);
                diff(q(t),t);
                diff(r(t),t);
                diff(alpha1(t),t,2)];

%% 坐标系定义与转换
% 这部分代码定义了机械臂和机器人本体之间的坐标系关系。

rM = [rMx; rMy; rMz]; % rM表示机械臂基座（即机械臂与机器人本体的连接点）相对于机器人本体原点的位置向量。

% R01是一个旋转矩阵，用于将机械臂基坐标系转换到机械臂动坐标系
R01 = [1, 0, 0; 0, cos(alpha1(t)), -sin(alpha1(t)); 0, sin(alpha1(t)), cos(alpha1(t))]; %
1->0

RV0 = [0, 1, 0; 1, 0, 0; 0, 0, -1]; % 0->V

RV1 = RV0 * R01; % 1->V Manipulator1 to vehicle

R1V = transpose(RV1); % V->1 vehicle to manipulator1

% 这对计算机臂的相对角速度非常关键。
x1 = RV1 * [1; 0; 0]; % z1是机械臂的旋转轴（局部Z轴）在机器人本体坐标系中的方向向量。注意 是方向向量，其表示了角速度的方向

rL1 = RV0 * [0; L1; 0]; % [0; 0; L1]表示机械臂的质心或关节相对于其基座在"局部坐标系"中的位置。rL1则是这个位置向量在"机器人本体坐标系"中的表示。

%% 速度转换
% 这部分使用刚体运动学的速度合成定理，计算机臂的速度。

% v1是机械臂的线速度，在机器人本体坐标系下表示。
% 它由两部分组成：
% 1. vv：机器人本体原点的平移速度。
% 2. cross(omegav, rM + rL1)：机器人本体旋转引起的，机械臂关节的附加速度。
% (rM + rL1) 是机械臂关节(或质心)相对于机器人本体原点的完整位置向量。该案例中质心与关节重合了

```



```

v1 = vv + cross(omegav, rM + rL1);

% omega1是机械臂的角速度，在机器人本体坐标系下表示。
% 它由两部分组成：
% 1. omegav：机器人本体的角速度。
% 2. z1 * diff(alpha1(t), t)：机械臂关节相对本体的旋转速度。
% diff(alpha1(t), t)是关节角度alpha1对时间的导数（即关节的旋转角速度）。
% % z1是这个旋转角速度的方向向量。
omega1 = omegav + x1 * diff(alpha1(t), t);

% ---

% 为什么最后几步又要乘以R1V?
% v11 = R1V * v1;：这行代码将机械臂的线速度从"本体坐标系" 转换回 "机械臂的局部坐标系（Frame 1）"。
% omega11 = R1V * omega1;：这行代码将机械臂的角速度从 "本体坐标系" 转换回 "机械臂的局部坐标系（Frame 1）"。
% 这么做的目的是为了确保机械臂的速度向量`v1`和其惯性矩阵M_1位于同一个坐标系下。
% * v1.' * M_1 * v1`所必需的。 即计算动能的时候，必须用围绕着自身的速度来计算动能
v11 = R1V * v1; %v->1
omega11 = R1V * omega1;
v1 = [v11; omega11];

%% 计算动能
T = 0.5 * nu.' * M_RB * nu + 0.5 * v1.' * M_1 * v1;

%% 求解gama-1
% 初始化三维数组来存储结果
AQvector = sym(zeros(7, 7, 7));

% 遍历矩阵 A 的每个元素并求偏导数
for i = 1:7
    for j = 1:7
        % 获取矩阵 A 的 (i, j) 元素
        A_ij = A(i, j);
        % 遍历广义坐标向量 q_vec 的每个元素，并求偏导数
        for k = 1:7
            AQvector(i, j, k) = diff(A_ij, q_vec(k));
        end
    end
end

gama1 = sym(zeros(7,7));

for i = 1:7
    for j = 1:7
        % 获取 AQvector 的切片，即 A(i,j) 对所有广义坐标的偏导数组成的列向量。
        % 在 MATLAB 中，这对应于 AQvector(i, j, :)。
        AQ_slice = squeeze(AQvector(i, j, :));

        % x1 的每个元素都是一个向量-矩阵-向量乘积。
        % 注意: nu_total 是 1x7, Transpose(J) 是 7x7, AQ_slice 是 7x1。
        % 这三者相乘的结果是一个标量。
        gama1(i, j) = nu_total.' * J.' * AQ_slice;
    end
end

```

```

end
end
% disp(gama1)
%% 求解gama-2
DAq1 = diff(A,q1(t));
DAq2 = diff(A,q2(t));
DAq3 = diff(A,q3(t));
DAq4 = diff(A,q4(t));
DAq5 = diff(A,q5(t));
DAq6 = diff(A,q6(t));
DAq7 = diff(A,alpha1(t));
DAq_eta = cat(3,DAq1,DAq2,DAq3,DAq4,DAq5,DAq6,DAq7);
gama2_l1 = nu_total.' * J.' * DAq1;
gama2_l2 = nu_total.' * J.' * DAq2;
gama2_l3 = nu_total.' * J.' * DAq3;
gama2_l4 = nu_total.' * J.' * DAq4;
gama2_l5 = nu_total.' * J.' * DAq5;
gama2_l6 = nu_total.' * J.' * DAq6;
gama2_l7 = nu_total.' * J.' * DAq7;
gama2 = [gama2_l1;
         gama2_l2;
         gama2_l3;
         gama2_l4;
         gama2_l5;
         gama2_l6;
         gama2_l7;];
%% 求gamma
gama = gama1 - gama2;
disp('gama is:')
disp(gama);
% H1 的计算
H1_temp = expand(diff(diff(T, u(t)), t));

H1 = simplify(jacobian(H1_temp,nu_dot_total));

H1_rest = simplify(H1_temp - H1*nu_dot_total);%H1_rest表示 拉格朗日法动力学建模求解的中间项， 其实
也就是惯性力和科氏力矩阵C的一部分

% H2 的计算
H2_temp = expand(diff(diff(T, v(t)), t));

H2 = simplify(jacobian(H2_temp,nu_dot_total));

H2_rest = simplify(H2_temp - H2*nu_dot_total);

% H3 的计算
H3_temp = expand(diff(diff(T, w(t)), t));

H3 = simplify(jacobian(H3_temp,nu_dot_total));

H3_rest = simplify(H3_temp - H3*nu_dot_total);

% H4 的计算

```

```

H4_temp = expand(diff(diff(T, p(t)), t));

H4 = simplify(jacobian(H4_temp, nu_dot_total));

H4_rest = simplify(H4_temp - H4*nu_dot_total);

% H5 的计算
H5_temp = expand(diff(diff(T, q(t)), t));

H5 = simplify(jacobian(H5_temp, nu_dot_total));

H5_rest = simplify(H5_temp - H5*nu_dot_total);

% H6 的计算
H6_temp = expand(diff(diff(T, r(t)), t));

H6 = simplify(jacobian(H6_temp, nu_dot_total));

H6_rest = simplify(H6_temp - H6*nu_dot_total);

% H7 的计算
H7_temp = expand(diff(diff(T, diff(alpha1(t), t)), t));

H7 = simplify(jacobian(H7_temp, nu_dot_total));

H7_rest = simplify(H7_temp - H7*nu_dot_total);

H = [H1; H2; H3; H4; H5; H6; H7];

disp('H:')
disp(H)
H_rest_total = [H1_rest;
                 H2_rest;
                 H3_rest;
                 H4_rest;
                 H5_rest;
                 H6_rest;
                 H7_rest];

test_sum = H - H.';
disp('test_sum: ')
disp(test_sum)
%% 求矩阵C(v, v_dot)
% 计算动能对类速度向量偏导
dTdvv = jacobian(T, nu_total);
JTgama = J.'*gama;
dTdeta = jacobian(T, q_vec);
% 第一C_nu分量
C_RB_1 = simplify(H_rest_total);
% 第二C_nu分量
C_RB_2 = simplify(JTgama * dTdvv.);
% 第三C_nu分量
C_RB_3 = simplify(J.' * dTdeta.) ;

```

```

C_nu = simplify(C_RB_1 + C_RB_2 + C_RB_3);
C = sevendof_C_decouple(C_nu); %
C = simplify(C);
disp(C)
%验证部分，不等于0，说明用简单的jacobian偏微分起不到等同的效果
C_test = simplify(jacobian(C_nu,nu_total));
C_sum_test = simplify(C_test - C*nu_total);
disp(C_sum_test);

%% 重力
Position_G = [xG,yG,zG];
Position_B = [xB,yB,zB];
G = gravity_with_collector(q4(t),q5(t),alpha1(t),W,W_c,B,Position_G,Position_B,L0,L1);
%% 水阻力项目，通过水动力参数识别获得。直接写数值
DL = diag([12,23,157,0.016,0.78,0.78,1e-7]); %线性阻尼
DV = diag([100,240,100,0.87,2.16,2.16,0]); %二次阻尼
D = (DL + DV)*nu_total; %体坐标系
D_eta = ((J.')\D)./J; %惯性坐标系

```

环境力建模(Disturbance)

💡 Tip

主要在算法里使用

搭载了采集器的机器人需要考虑环境力(Ocean Current) 还有 采集时的接触力的影响

Ocean & Current

```

function tau_env = disturbance_7dof(t, nu_k, J_inv_k_full, Z_depth, DL_env)
    %% 环境扰动参数
    % 洋流速度，惯性坐标系下
    NU_C_INERTIAL = [1; 0.8; 0.2; 0; 0; 0];
    % 波浪力参数
    A_WAVE = 5; % 波浪力幅值 N
    OMEGA_WAVE = 0.5; % 波浪频率 rad/s
    K_WAVE_DECAY = 1.0; % 深度衰减系数
    % 随机噪声参数
    K_NOISE = diag([2, 2, 1, 0.2, 0.2, 0.2]);
    %% 初始化
    tau_env = zeros(6, 1); % 体坐标系下的6DOF环境扰动
    %% 1. 洋流扰动
    % 惯性系洋流速度转换到体坐标系
    nu_c_body = J_inv_k_full(1:6, 1:6) * NU_C_INERTIAL;
    % 相对速度
    nu_r = nu_k - nu_c_body;
    % 洋流造成的阻力
    tau_current = -DL_env * nu_r;
    tau_env = tau_env + tau_current;
    %% 2. 波浪扰动
    % 深度越深，波浪影响越小
    wave_decay = exp(-K_WAVE_DECAY * abs(Z_depth));

```

```

wave_force_x = A_WAVE * sin(OMEGA_WAVE * t) * wave_decay;
wave_force_y = A_WAVE * cos(OMEGA_WAVE * t) * wave_decay;
tau_wave_body = [wave_force_x; wave_force_y; 0; 0; 0; 0];
tau_env = tau_env + tau_wave_body;
%% 3. 随机噪声
tau_noise = K_NOISE * randn(6, 1);
tau_env = tau_env + tau_noise;

```

end

Contact

💡 Tip

采集的时候和土壤接触，可以参考**宾汉姆模型**。

```

function tau_contact = contact_force(nu_k, Z_depth)
    tau_contact = zeros(6, 1); % 驱动杆在内部，所以才采集接触力基本不会影响采集器的开合，主要是接触的反
    作用力会作用到机器人本体上
    u_body = nu_k(1);
    %% 补充宾汉姆体模型参数（基于文献实验数据）
    % 文献来源: Study of the resistance on the side surface of the underwater vehicle's soil
    base % (Ananiev, 2023)
    % --- 土壤参数（使用 Model No. 1 的规范值作为保守假设） ---
    tau_l = 0.4 * 1000; % 极限剪切应力 tau_l1 = 0.4 kPa, 转换为 Pa (N/m^2) [cite:
    179]
    eta_viscosity = 1.20e-3 * 1000; % 黏度系数 eta_1 = 1.20e-3 kPa·s, 转换为 Pa·s [cite: 128]
    delta_shear_layer = 0.02; % 剪切边界层厚度 delta = 20 mm, 转换为米 [cite: 77]
    Z_contact_threshold = 0.25; % 接触深度阈值 m
    A_MV = 0.02; % 假设采集工具的侧表面积 A_MV 等于 MV 模型 No. 1 的侧面积 0.02 m^2 [cite: 84]
    Z_CONTACT_OFFSET = -0.8; % 接触点到CG的垂直距离 (m)。
    %% --- 随机扰动范围 ---
    % 引入随机性到黏度系数，模拟土壤不确定性
    eta_rand_factor = [0.8, 1.5]; % 假设黏度系数在标称值的 80% 到 150% 之间随机变化

    % 检查是否满足采集条件: Z_depth >= 0.25 (接触) 且 u_body (有速度)
    if Z_depth >= Z_contact_threshold && abs(u_body) > 1e-3

        % 1. 宾汉姆体模型计算 (F_x, 第 1 维):
        % a. 计算剪切变形率 gamma_dot
        % 剪切变形率 gamma_dot = 速度 v / 边界层厚度 delta [cite: 100]
        gamma_dot = abs(u_body) / delta_shear_layer;

        % b. 引入随机黏度系数（模拟土壤不确定性）
        % rand_eta 介于 0.8*eta_viscosity 和 1.5*eta_viscosity 之间
        rand_factor = eta_rand_factor(1) + (eta_rand_factor(2) - eta_rand_factor(1)) *
        rand(1);
        eta_k_soil = eta_viscosity * rand_factor;

        % c. 计算剪切应力 tau_i = tau_l + eta * gamma_dot [cite: 93]
        tau_i = tau_l + eta_k_soil * gamma_dot; % 结果单位是 Pa (N/m^2)

        % d. 计算总接触阻力 F_x (力 = 应力 * 面积)
    end

```

```

F_x_contact_mag = A_MV * tau_i; % 接触力大小 (N)

% 确保接触力方向与运动方向相反
F_x_contact = -sign(u_body) * F_x_contact_mag;
tau_contact(1) = F_x_contact;

% 2. 俯仰力矩 (M_y, 第 5 维): 力矩耦合到接触力
% tau_contact(5) = 偏置 Z_CONTACT_OFFSET * 接触力 F_x
tau_contact(5) = Z_CONTACT_OFFSET * F_x_contact;
end
end

```

推力分配

QUAD-With-Collector-Distribution

```

function A_thruster = QUAD_with_collector_dist(gama,beta,L,R,k)
%% 推力分配
%注意推进器的分布 后面从左往右12 前面从左往右34 然后中间从左往右56 最后一个7
param1 = cos(gama);
param2 = sin(gama);
param3 = cos(beta);
param4 = sin(beta);
A_thruster = [-param2*param3, -param2*param3, param2*param3, param2*param3, 0, 0, 0, 0;
               param1*param3, -param1*param3, param1*param3, -param1*param3, 0, 0, 0, 0;
               param4, param4, param4, param4, 1, 1, 1, 0;
               L*param4, -L*param4, L*param4, -L*param4, R, -R, 0, 0;
               L*param4, L*param4, -L*param4, -L*param4, 0, 0, R, 0;
               k*param3*param1, -k*param3*param1, -k*param3*param1, k*param3*param1, 0, 0,
0,
               0, 0, 0, 0, 0, 0, 0, 0]; % 7x8 矩阵 7个推进器 一个被动关节
end

```